

Paso 9 - Métodos | Solucionario

Las cinco soluciones del paso 9. La pregunta que hay que hacerse en todas: ¿este método **hace** algo (`void`) o **responde** algo (`return`)?

Úsalo después de intentarlo. Cada solución viene con la salida real del programa: si la tuya coincide, está bien aunque el código sea distinto. Los `.java` de esta carpeta son los mismos que se muestran abajo, listos para ejecutar: `cd soluciones/paso09` y `javac *.java`.

B9.1 - Los tres clásicos

Dos métodos que **responden** (`boolean` y `double`) y uno que **hace** (`void`).

▼ Ver la solución (B9_1_TresMetodos.java)

B9_1_TresMetodos.java

```
public class B9_1_TresMetodos {

    static boolean esPar(int n) {
        return n % 2 == 0;
    }

    static double areaCirculo(double radio) {
        return Math.PI * radio * radio;
    }

    static void saludar(String nombre, int veces) {
        for (int i = 1; i <= veces; i++) {
            System.out.println("Hola, " + nombre);
        }
    }

    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (esPar(i)) {
                System.out.println(i + " es par");
            } else {
                System.out.println(i + " es impar");
            }
        }

        System.out.printf("Area de radio 2: %.2f%n", areaCirculo(2));
        saludar("Ana", 3);
    }
}
```

Salida:

```
1 es impar
2 es par
3 es impar
4 es par
5 es impar
6 es par
7 es impar
8 es par
9 es impar
10 es par
Area de radio 2: 12.57
Hola, Ana
Hola, Ana
Hola, Ana
```

Lo que importa. `esPar` devuelve **directamente la comparación**: `return n % 2 == 0;`. No hace falta escribir `if (...) return true; else return false;` — eso ya es un `boolean`. Si lo escribiste con `if`, funciona igual, pero mira las dos versiones juntas: la corta dice lo mismo sin ruido.

Y el que devuelve no imprime. `areaCirculo` no tiene ningún `println` adentro: devuelve el número y el `main` decide qué hacer con él. Por eso lo pude usar dentro de un `printf`. Un método que imprime solo sirve para imprimir; uno que devuelve sirve para todo.

B9.2 - La nota en palabras

El mismo `if` del B5.6, pero **devolviendo** el texto en vez de imprimirlo.

▼ Ver la solución (B9_2_Clasificar.java)

B9_2_Clasificar.java

```
public class B9_2_Clasificar {

    static String clasificar(double nota) {
        if (nota < 1.0 || nota > 7.0) {
            return "Esa nota no existe";
        } else if (nota >= 6.5) {
            return "Excelente";
        } else if (nota >= 5.5) {
            return "Muy buena";
        } else if (nota >= 4.0) {
            return "Suficiente";
        } else {
            return "Insuficiente";
        }
    }

    public static void main(String[] args) {
        double[] notas = {5.5, 3.2, 6.8, 4.0};

        for (double nota : notas) {
            System.out.println(nota + " -> " + clasificar(nota));
        }
    }
}
```

Salida:

```
5.5 -> Muy buena
3.2 -> Insuficiente
6.8 -> Excelente
4.0 -> Suficiente
```

Lo que importa. Cada rama termina en `return`, así que el método sale apenas encuentra su caso. Por eso **no hacen falta** los `else ...` pero escribirlos deja clarísimo que son alternativas excluyentes. Las dos versiones compilan; esta se lee mejor.

Si borras el último `else` y dejas solo `return "Insuficiente";` al final, también compila. Pero si borras el `return` final, `javac` dice `missing return statement`: Java exige que **todos** los caminos

devuelvan algo, incluso los que tú sabes que no pueden pasar.

Este método es la mitad del B13.2: un método que devuelve texto se puede imprimir, guardar en un arreglo o comparar. Uno que imprime, no.

B9.3 - La tabla, ahora como método

El `for` del paso 7 metido en un método, y una segunda versión con un parámetro más. Eso último ya es **sobrecarga**.

▼ Ver la solución (B9_3_TablaMetodo.java)

B9_3_TablaMetodo.java

```
public class B9_3_TablaMetodo {

    static void tabla(int n) {
        tabla(n, 10);           // la version corta llama a la larga
    }

    static void tabla(int n, int hasta) {
        System.out.println("Tabla del " + n + ":");
        for (int i = 1; i <= hasta; i++) {
            System.out.printf("%d x %2d = %3d%n", n, i, n * i);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        tabla(3);
        tabla(7, 3);
        tabla(9, 5);
    }
}
```

Salida:

Tabla del 3:

```
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
```

Tabla del 7:

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
```

Tabla del 9:

```
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
```

Lo que importa. Los dos métodos se llaman igual y Java no se confunde: elige por la **cantidad de parámetros**. Eso es la sobrecarga del paso 12, y aquí la usaste sin que nadie te la explicara.

El truco de `tabla(n)` llamando a `tabla(n, 10)` evita repetir el `for`. Si mañana cambias el formato de la tabla, lo cambias en **un** lugar. Es exactamente lo que hace `línea()` en el paso 12.

Prueba a escribir el `for` completo en las dos versiones y después cambia `%3d` por `%4d` en una sola: la tabla del 3 y la del 7 quedan distintas. Ese es el costo de duplicar código, y se paga siempre.

B9.4 - Partir un programa en métodos

El B8.1 repartido en cinco métodos. El `main` queda en cuatro líneas y se lee como un resumen de lo que hace el programa.

▼ Ver la solución (B9_4_NotasEnMetodos.java)

B9_4_NotasEnMetodos.java

```
import java.util.Arrays;
import java.util.Scanner;

public class B9_4_NotasEnMetodos {

    static double[] leerNotas(Scanner sc, int cantidad) {
        double[] notas = new double[cantidad];
        for (int i = 0; i < notas.length; i++) {
            System.out.print("Nota " + (i + 1) + ": ");
            notas[i] = sc.nextDouble();
        }
        return notas;
    }

    static double promedio(double[] notas) {
        double suma = 0;
        for (double n : notas) {
            suma += n;
        }
        return suma / notas.length;
    }

    static double maxima(double[] notas) {
        double mayor = notas[0];
        for (double n : notas) {
            if (n > mayor) {
                mayor = n;
            }
        }
        return mayor;
    }

    static int contarRojas(double[] notas) {
        int rojas = 0;
        for (double n : notas) {
            if (n < 4.0) {
                rojas++;
            }
        }
        return rojas;
    }
}
```

```

static void imprimirInforme(double[] notas) {
    System.out.println();
    System.out.println("Notas: " + Arrays.toString(notas));
    System.out.printf("Promedio: %.1f%n", promedio(notas));
    System.out.println("Nota mas alta: " + maxima(notas));
    System.out.println("Notas rojas: " + contarRojas(notas));
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    double[] notas = leerNotas(sc, 5);
    imprimirInforme(notas);
    sc.close();
}
}

```

Salida:

```

Nota 1: 5.5
Nota 2: 6.2
Nota 3: 3.8
Nota 4: 7.0
Nota 5: 4.1

Notas: [5.5, 6.2, 3.8, 7.0, 4.1]
Promedio: 5.3
Nota mas alta: 7.0
Notas rojas: 1

```

Lo que importa. Lee el `main`: *lee las notas, imprime el informe*. En cuatro líneas se entiende **qué** hace el programa sin saber **cómo** lo hace. Eso es todo lo que se gana partiendo en métodos, y es la idea que sostiene el resto del curso.

`imprimirInforme` **llama a los otros tres** en vez de recalcular. Si mañana cambias qué es una nota roja, tocas `contarRojas` y listo.

Un método puede devolver un arreglo: `leerNotas` devuelve `double[]`. El tipo de retorno puede ser cualquier cosa, incluidos arreglos y, desde la demo 1, tus propios objetos.

Compara este archivo con tu B8.1: hace exactamente lo mismo. La diferencia es que estos cinco métodos **se pueden mudar a una clase aparte**, y eso es literalmente el paso 10.

B9.5 - Desafío: métodos que se llaman entre ellos

`contarPrimos` no repite el código de `esPrimo`: lo **usa**.

▼ Ver la solución (B9_5_Primos.java)

B9_5_Primos.java

```
public class B9_5_Primos {

    static boolean esPrimo(int n) {
        if (n < 2) {
            return false;
        }
        for (int divisor = 2; divisor < n; divisor++) {
            if (n % divisor == 0) {
                return false;          // encuentro un divisor: no es primo
            }
        }
        return true;
    }

    static int contarPrimos(int hasta) {
        int cuantos = 0;
        for (int n = 2; n <= hasta; n++) {
            if (esPrimo(n)) {
                cuantos++;
            }
        }
        return cuantos;
    }

    public static void main(String[] args) {
        System.out.println("Primos hasta 30: " + contarPrimos(30));
        System.out.println("Primos hasta 100: " + contarPrimos(100));
        System.out.println("Es primo el 97? " + esPrimo(97));
        System.out.println("Es primo el 1? " + esPrimo(1));
    }
}
```

Salida:

```
Primos hasta 30: 10
Primos hasta 100: 25
Es primo el 97? true
Es primo el 1? false
```

La respuesta: un solo método. Si mañana cambias `esPrimo` (por ejemplo, para buscar divisores solo hasta la raíz cuadrada), `contarPrimos` se vuelve más rápido **sin tocarlo**. Eso es lo que ganas al no duplicar: el conocimiento sobre qué es un primo vive en **un** lugar.

Fíjate en el `return false` dentro del `for`. Reemplaza a la bandera `boolean` y al `break` del B7.5: cuando estás dentro de un método, `return` corta todo de una vez. Compara las dos versiones, es el mismo algoritmo escrito mucho más corto.

Y el `if (n < 2)`: el 1 no es primo, y sin esa línea `esPrimo(1)` devolvería `true`, porque el `for` no da ninguna vuelta y llega directo al `return true`. Los casos borde (0, 1, negativos) son siempre lo primero que hay que pensar al escribir un método que responde `boolean`.

Volver a la [hoja del paso 9](#) o al [paso 9 de la guía](#).